MONET – Supplementary Material

Algorithm, Hyperparameters, Sensitivity Analysis, and Node-Coverage Bounds

Julian Hatzky Thomas Bartz-Beielstein A. E. Eiben Anil Yaman

This document accompanies the MONET paper *"Multi-Task Optimization over Networks of Tasks"* (Hatzky et al., 2026) and the code repository at `https://github.com/ju2ez/MONET`. It collects the full MONET pseudocode (main loop + the two learning subroutines), the hyperparameter reference used throughout the experiments, the per-domain sensitivity analysis (SHAP beeswarm plots and SPOT-tuned configurations), and the coupon-collector bounds that justify the 10^6-evaluation budget.

Contents

1 MONET Algorithm **1**

2 MONET Hyperparameter Descriptions **3**

3 Detailed Sensitivity Analysis **4**

3.1 Per-Domain SHAP Analysis 4

3.2 SPOT-Tuned Configurations 8

4 Node Coverage Guarantees Under Random Sampling **8**

1 MONET Algorithm

Algorithm 1 gives the main MONET loop; Algorithms 2 and 3 detail the two learning subroutines invoked from it.

Algorithm 1 MONET

Require: Task set $T = \{\boldsymbol{\tau}_1, \ldots, \boldsymbol{\tau}_n\}$, fitness function f, task distance d
Require: Budget E, init fraction K_{init}, neighborhood size N, selection strategy $\textsc{Select}(\cdot)$, probability p_{ind}
Ensure: Network $\mathcal{N}$ mapping each task to its best solution

$\mathcal{N}[\boldsymbol{\tau}] \leftarrow \emptyset$ for all $\boldsymbol{\tau} \in T$ ▷ empty network
for each $\boldsymbol{\tau}_i \in T$ **do** ▷ build k-NN task graph
  $\text{Nb}(\boldsymbol{\tau}_i) \leftarrow N$ nearest tasks under d
end for

for $i \leftarrow 1$ **to** K_{init} **do** ▷ random initialisation
  $\boldsymbol{\tau} \leftarrow \textsc{RandomTask}(T)$
  $\mathbf{x} \sim \mathcal{U}(p_{\min}, p_{\max})$
  $f_x \leftarrow f(\mathbf{x}, \boldsymbol{\tau})$
  if $\mathcal{N}[\boldsymbol{\tau}] = \emptyset$ **or** $f_x \geq \mathcal{N}[\boldsymbol{\tau}].\text{fitness}$ **then** ▷ elitist update
    $\mathcal{N}[\boldsymbol{\tau}] \leftarrow (\mathbf{x}, f_x)$
  end if
end for

$e \leftarrow K_{\text{init}}$ ▷ main loop
while $e < E$ **do**
  $\boldsymbol{\tau} \leftarrow \textsc{RandomTask}(T)$ ▷ sample focal task
  if $\text{rand}() < p_{\text{ind}}$ **then** ▷ individual learning
    $\textsc{IndividualLearning}(\boldsymbol{\tau}, \mathcal{N})$ ▷ Alg. 2
    $e \leftarrow e + 1$
  else ▷ social learning
    $\boldsymbol{\tau}_j \leftarrow \textsc{Select}(\text{Nb}(\boldsymbol{\tau}))$ ▷ pick one neighbor
    if $\boldsymbol{\tau}_j \neq \text{nil}$ **then** ▷ skip if no graph neighbor exists
      $\textsc{SocialLearning}(\boldsymbol{\tau}, \boldsymbol{\tau}_j, \mathcal{N})$ ▷ Alg. 3
      $e \leftarrow e + 1$
    end if
  end if
end while
return $\mathcal{N}$

Individual learning. When individual learning is selected, the current elite at node $\boldsymbol{\tau}_i$ is mutated independently of any neighbor. If no solution exists at the node, a random solution is drawn from $\mathcal{U}(p_{\min}, p_{\max})$ and evaluated. Otherwise, Gaussian mutation with standard deviation σ_m scaled by the parameter range is applied. The offspring replaces the current elite only if its fitness is at least as high.

Algorithm 2 Individual Learning

Require: Task $\boldsymbol{\tau}$, network $\mathcal{N}$, bounds $p_{\min}, p_{\max}$, mutation std σ_m
Ensure: Updated $\mathcal{N}[\boldsymbol{\tau}]$

```
if N[τ] = ∅ then                                   ▷ initialise empty node
    x ~ U(p_min, p_max)                            ▷ random solution
    N[τ] ← (x, f(x, τ))
    return
end if
x ← N[τ].solution                                  ▷ current elite
ε ~ N(0, σ_m(p_max − p_min))                       ▷ Gaussian perturbation
y ← clip(x + ε, p_min, p_max)                      ▷ mutant offspring
f_y ← f(y, τ)
if f_y ≥ N[τ].fitness then                         ▷ elitist replacement
    N[τ] ← (y, f_y)
end if
```

Social learning. When social learning is selected, the current elite is recombined with a solution drawn from a neighboring node. If the current node is empty, the neighbor's solution is copied as an initial seed (or a random solution is used when no neighbor has a solution). Recombination is performed via Simulated Binary Crossover (SBX; Deb & Agrawal, 1995) with distribution index $\eta_c = 10$ (Deb, 2001). As with individual learning, the offspring replaces the elite only upon improvement.

Algorithm 3 Social Learning

Require: Task $\boldsymbol{\tau}$, neighbor task $\boldsymbol{\tau}_j$, archive $\mathcal{A}$, bounds $p_{\min}, p_{\max}$
Ensure: Updated $\mathcal{A}[\boldsymbol{\tau}]$

```
if A[τ] = ∅ then                                   ▷ initialise empty node
    if A[τ_j] ≠ ∅ then
        x ← A[τ_j].solution                        ▷ copy from neighbor
    else
        x ~ U(p_min, p_max)                        ▷ random fallback
    end if
    A[τ] ← (x, f(x, τ))
    return
end if
if A[τ_j] = ∅ then
    return                                         ▷ no valid neighbor solution
end if
y ← SBX(A[τ].solution, A[τ_j].solution, η_c = 10)  ▷ crossover
y ← clip(y, p_min, p_max)
f_y ← f(y, τ)
if f_y ≥ A[τ].fitness then                         ▷ elitist replacement
    A[τ] ← (y, f_y)
end if
```

2 MONET Hyperparameter Descriptions

Table 1 summarises the four hyperparameters varied in the sensitivity analysis. All other algorithmic parameters (mutation operator, crossover operator, initialisation strategy) are held fixed throughout.

Table 1: MONET hyperparameters and their roles.

Parameter	Values	Description
p_{ind}	0, 0.3, 0.5, 0.7, 1	Probability that a given evaluation step performs *individual learning* (mutation of the focal task's own elite) rather than *social learning* (crossover with a neighbor's elite). At $p_{\text{ind}} = 0$ only social learning occurs; at $p_{\text{ind}} = 1$ only individual learning occurs.
strategy	random, best_fitness	Neighbor selection strategy during social learning. `random`: uniform random selection among all neighbors. `best_fitness`: deterministically select the neighbor with the highest current fitness.
neighborhood	random, closest, distance_prop.	Method for constructing the neighbor set of each task. `closest`: the N nearest tasks by Euclidean distance in task space. `random`: N tasks sampled uniformly at random (sorted by similarity after sampling). `distance_proportional`: N tasks sampled *without replacement* with probability proportional to task-space similarity $s(\boldsymbol{\tau}_i, \boldsymbol{\tau}_j) = 1/(1 + \lVert\boldsymbol{\tau}_i - \boldsymbol{\tau}_j\rVert)$; this softly biases the neighborhood toward nearby tasks while retaining a chance of including distant ones.
neighbor_pct.	1%, 5%, 10%	Fraction of the total task set used as the neighborhood size $N = \lceil \texttt{neighbor_percentage} \cdot \lvert\mathcal{V}\rvert \rceil$. For instance, with 5 000 tasks and 5%, each task has $N = 250$ neighbors. Controls graph sparsity: lower values yield sparser graphs with more localised knowledge transfer.

3 Detailed Sensitivity Analysis

This section provides per-domain SHAP beeswarm plots and the SPOT-tuned hyperparameter configurations that complement the global analysis in the main paper.

3.1 Per-Domain SHAP Analysis

Figures 2–4 show the SHAP beeswarm plots for the four domains. Each point represents one MONET run; its horizontal position indicates the SHAP value (contribution to the predicted metric), and its colour encodes the feature value (red = high, blue = low).

In all domains, p_{ind} exhibits the strongest effect: high values of p_{ind} (more individual learning) consistently decrease predicted fitness, corroborating the global finding that social learning is the primary driver of performance. The remaining parameters show smaller, more task-specific effects.

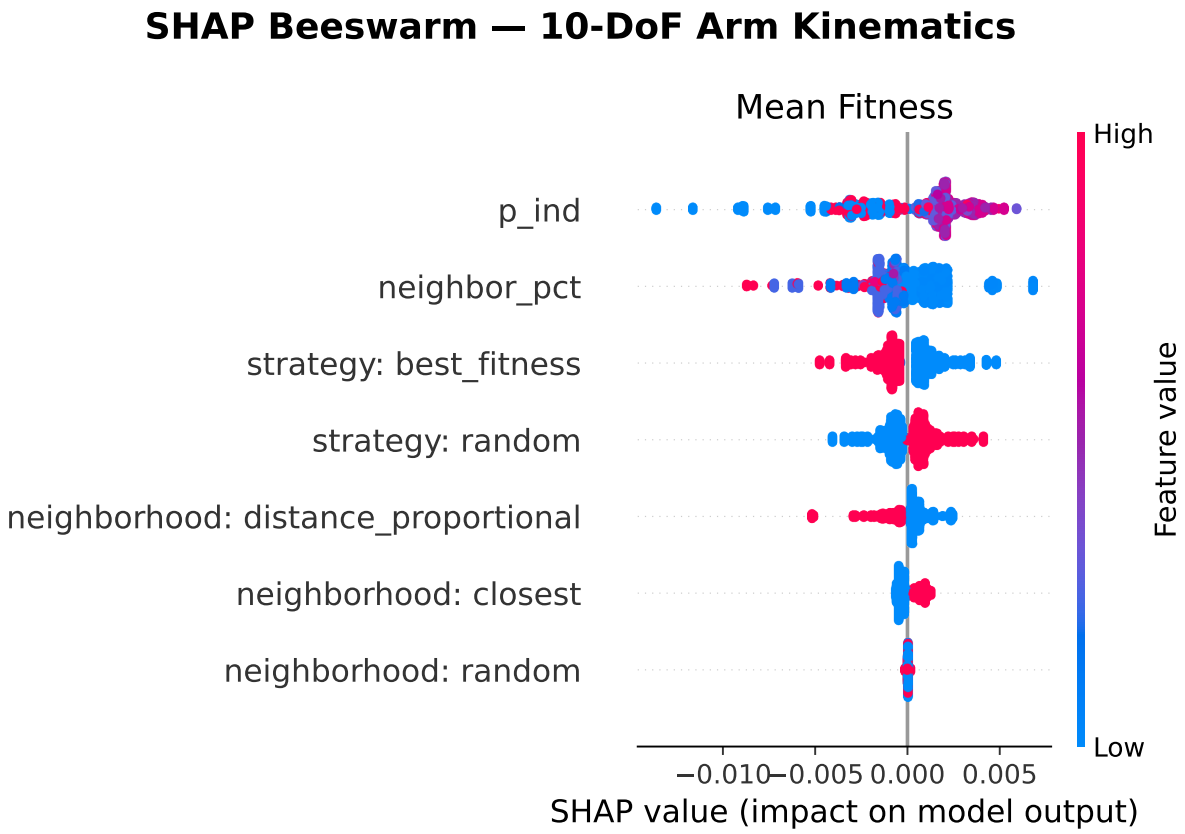

Figure 1: SHAP beeswarm plot for the 10-DoF Arm domain. Each row is a hyperparameter; each dot is a MONET run. Colour indicates the feature value (red = high, blue = low); horizontal position indicates the SHAP value.

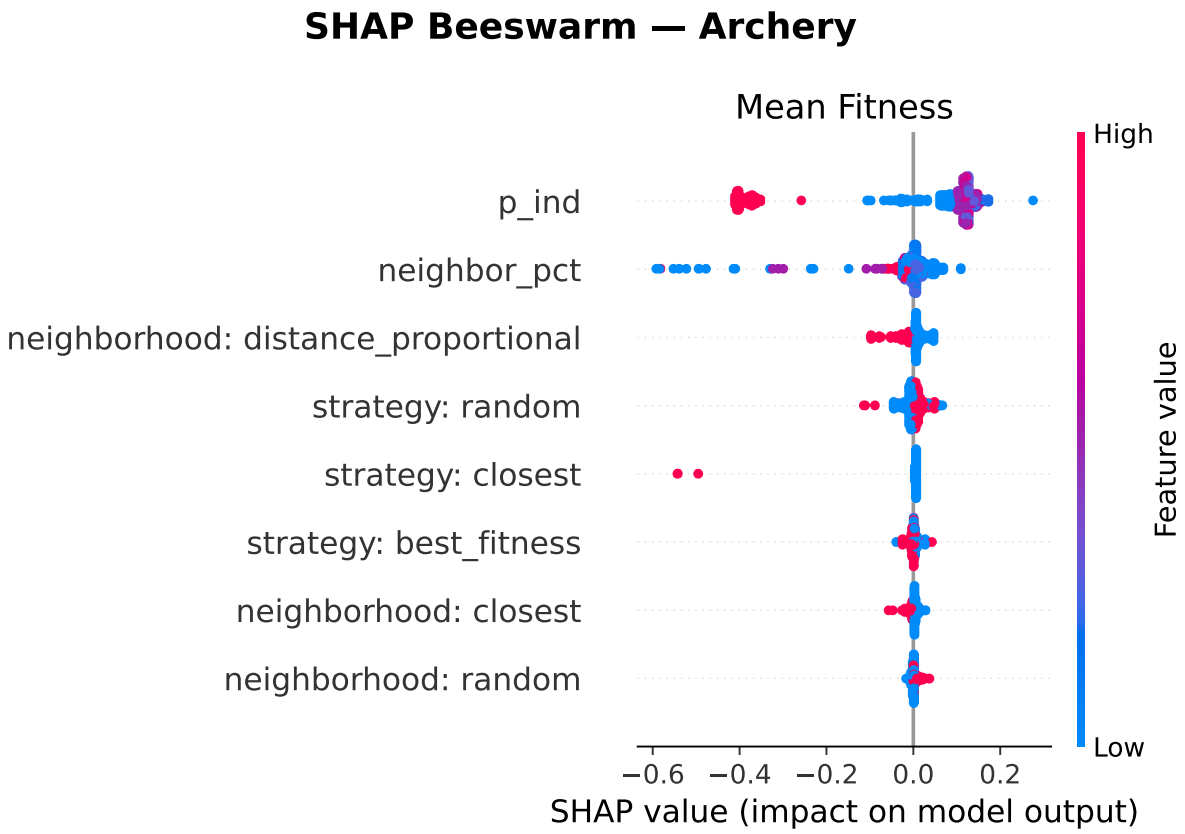

Figure 2: SHAP beeswarm plot for the Archery domain. Panels as in Figure 1.

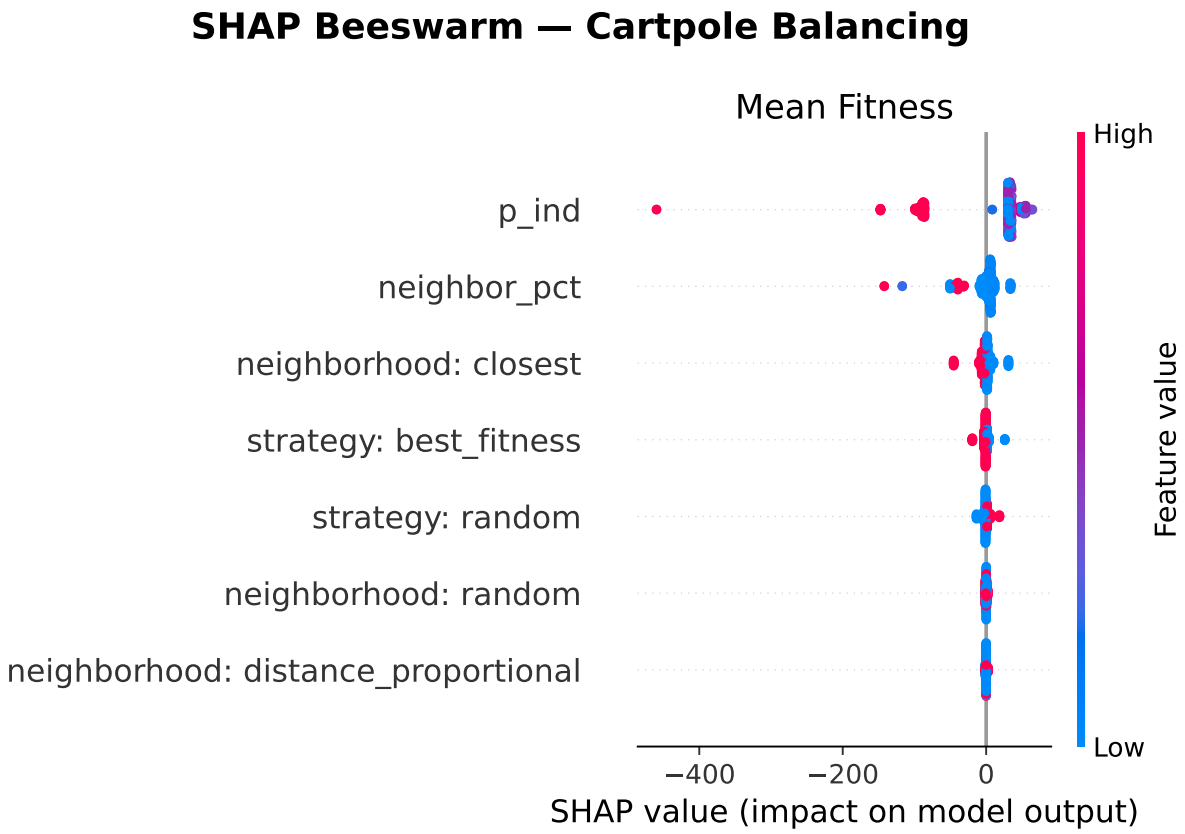

Figure 3: SHAP beeswarm plot for the Cartpole Balancing domain. Panels as in Figure 1.

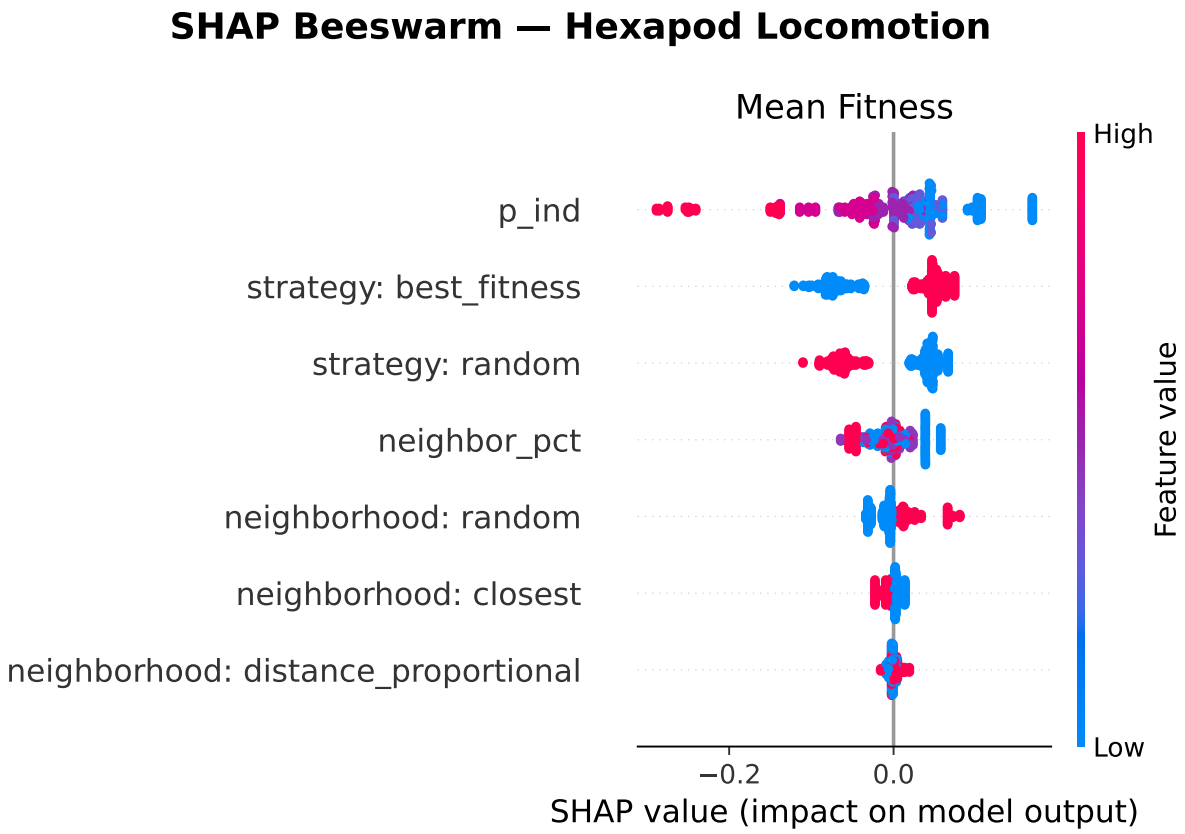

Figure 4: SHAP beeswarm plot for the Hexapod Locomotion domain. Panels as in Figure 1.

3.2 SPOT-Tuned Configurations

Table 2 lists the per-domain configurations found by the SPOT hyperparameter tuner (Bartz-Beielstein et al., 2010, 2024, 2026). The surrogate-model-based tuning loop is implemented in `hpt_spot.py` in the repository root and drives `run_algorithms.py` as a black-box evaluator.

Table 2: SPOT-tuned MONET configurations per domain.

Domain	**strategy**	**neighborhood**	p_{ind}	**neighbor_pct.**
Arm	random	closest	0.3	1%
Archery	best_fitness	closest	0.0	2%
Cartpole	best_fitness	random	0.5	1%
Hexapod	best_fitness	random	0.0	1%

4 Node Coverage Guarantees Under Random Sampling

If we draw at each evaluation step a random node and perform an action, we may ask how many evaluations are required so that we visit each of the ν nodes at least once with high certainty. This is the classical *coupon collector's problem* (Feller, 1968). Writing M for the number of draws, the expected number of steps needed to visit all ν nodes at least once is

$$\mathbb{E}[M] = \nu H_\nu = \nu \left(\ln \nu + \gamma + O\left(\frac{1}{\nu}\right) \right),$$

where H_ν is the ν-th harmonic number and $\gamma \approx 0.57721$ is the Euler-Mascheroni constant.

Beyond the expectation, one may also ask for a high-probability guarantee. Using the asymptotic limit for the coupon-collector completion time, the probability that all nodes have been visited by draw

$$M = \nu(\ln \nu + c)$$

converges to

$$\mathbb{P}\big(M \le \nu(\ln \nu + c)\big) \approx e^{-e^{-c}}.$$

To achieve at least 99% probability of having visited all nodes, we solve for c:

$$\begin{aligned} e^{-e^{-c}} &= p, \\ \ln\Big(e^{-e^{-c}}\Big) &= \ln(p), \\ -e^{-c} &= \ln(p), \\ \text{For } p = 0.99: \quad c &= -\ln\big(-\ln(0.99)\big) \approx 4.60015 \approx \ln(100). \end{aligned}$$

Since each node is selected independently with probability $\frac{1}{\nu}$ at every draw, the visit count of any particular node over M draws follows a Binomial distribution,

$$X_i \sim \text{Binomial}\big(M, \tfrac{1}{\nu}\big), \qquad \mathbb{E}[X_i] = \frac{M}{\nu}.$$

We evaluate four environments with differing numbers of distinct tasks and a fixed draw budget of $M = 1{,}000{,}000$ evaluations:

Table 3: Coupon-collector bounds for the four benchmark environments with $M = 1{,}000{,}000$ total evaluations.

Environment	ν	$\mathbb{E}[M]$	$M_{0.99}$	$\mathbb{E}[X_i]$
Hexapod	2 000	16 356	24 412	500
Archery / Arm / Cartpole	5 000	45 472	65 612	200

References (selected)

- Bartz-Beielstein, T. (2010). *SPOT: An R Package for Automatic and Interactive Tuning of Optimization Algorithms by Sequential Parameter Optimization.* arXiv:1006.4645 • doi:10.48550/arXiv.1006.4645.
- Bartz-Beielstein, T., Zaefferer, M., and Rehbach, F. (2024). *Hyperparameter Tuning for Machine and Deep Learning with R: A Practical Guide.* Springer. doi:10.1007/978-981-19-5170-1.
- Bartz-Beielstein, T. (2026). *Optimization with SpotOptim.* arXiv:2604.13672 • pypi.org/project/spotoptim.
- Deb, K. and Agrawal, R. B. (1995). *Simulated Binary Crossover for Continuous Search Space. Complex Systems* 9(2):115–148. complex-systems.com/abstracts/v09_i02_a02.
- Deb, K. (2001). *Multi-Objective Optimization Using Evolutionary Algorithms.* Wiley. ISBN 978-0-471-87339-6.
- Feller, W. (1968). *An Introduction to Probability Theory and Its Applications*, Vol. 1, 3rd ed. Wiley.
- Hutter, F., Hoos, H., and Leyton-Brown, K. (2014). *An Efficient Approach for Assessing Hyperparameter Importance.* In *Proc. ICML 2014*, PMLR 32:754–762. proceedings.mlr.press/v32/hutter14.
- Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., and Lee, S.-I. (2020). *From Local Explanations to Global Understanding with Explainable AI for Trees. Nature Machine Intelligence* 2(1):56–67. doi:10.1038/s42256-019-0138-9.

- Lundberg, S. M. and Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions*. In *Advances in Neural Information Processing Systems* 30 (NeurIPS 2017). arXiv:1705.07874 • NeurIPS proceedings.

- Mann, H. B. and Whitney, D. R. (1947). *On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other*. *Annals of Mathematical Statistics* 18(1):50–60. doi:10.1214/aoms/1177730491.

- Holm, S. (1979). *A Simple Sequentially Rejective Multiple Test Procedure*. *Scandinavian Journal of Statistics* 6(2):65–70. jstor:4615733.